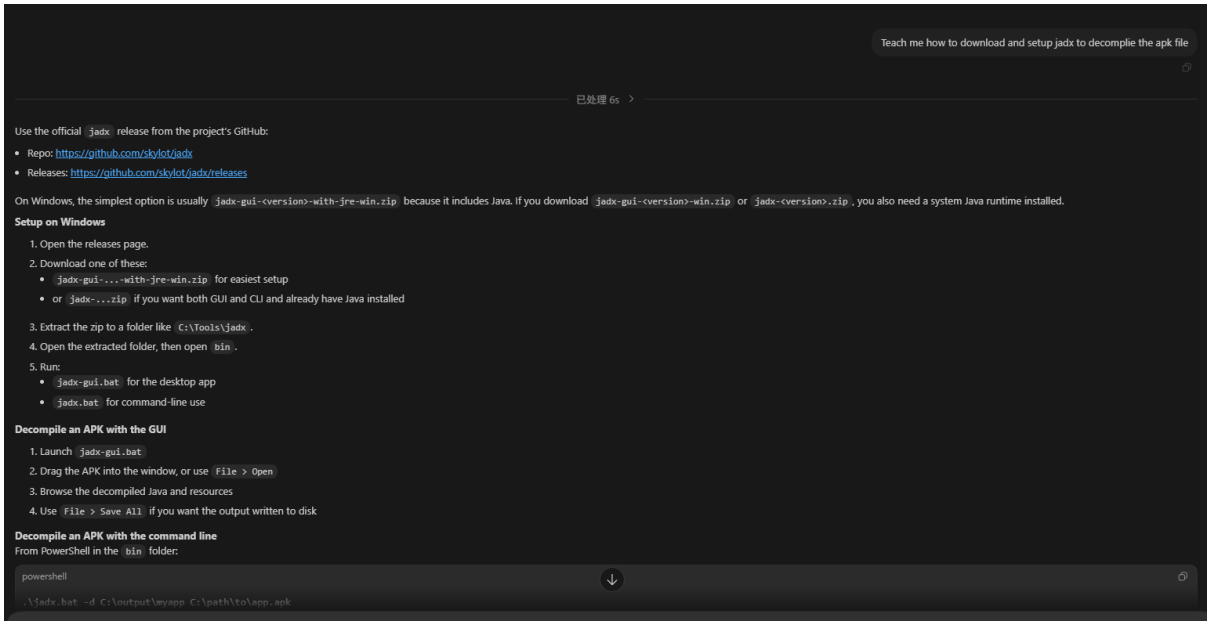
1. Task 1: Unpack and Decompile the APK • In your own words, define what an APK is and why decompilation is needed for auditing.

   <u>An APK is a compressed app. After decompilation, people can read the code and find the vulnerability for auditing.</u>

   • Use AI to choose/setup decompilation tooling (for example, jadx, apk-tools). Record at least one step-by-step AI interaction



Teach me how to download and setup jadx to decomplie the apk file

已处理 6s

Use the official `jadx` release from the project's GitHub:

- Repo: https://github.com/skylot/jadx
- Releases: https://github.com/skylot/jadx/releases

On Windows, the simplest option is usually `jadx-gui-<version>-with-jre-win.zip` because it includes Java. If you download `jadx-gui-<version>-win.zip` or `jadx-<version>.zip`, you also need a system Java runtime installed.

Setup on Windows

1. Open the releases page.
2. Download one of these:
   - `jadx-gui-...-with-jre-win.zip` for easiest setup
   - or `jadx-...zip` if you want both GUI and CLI and already have Java installed
3. Extract the zip to a folder like `C:\Tools\jadx`.
4. Open the extracted folder, then open `bin`.
5. Run:
   - `jadx-gui.bat` for the desktop app
   - `jadx.bat` for command-line use

Decompile an APK with the GUI

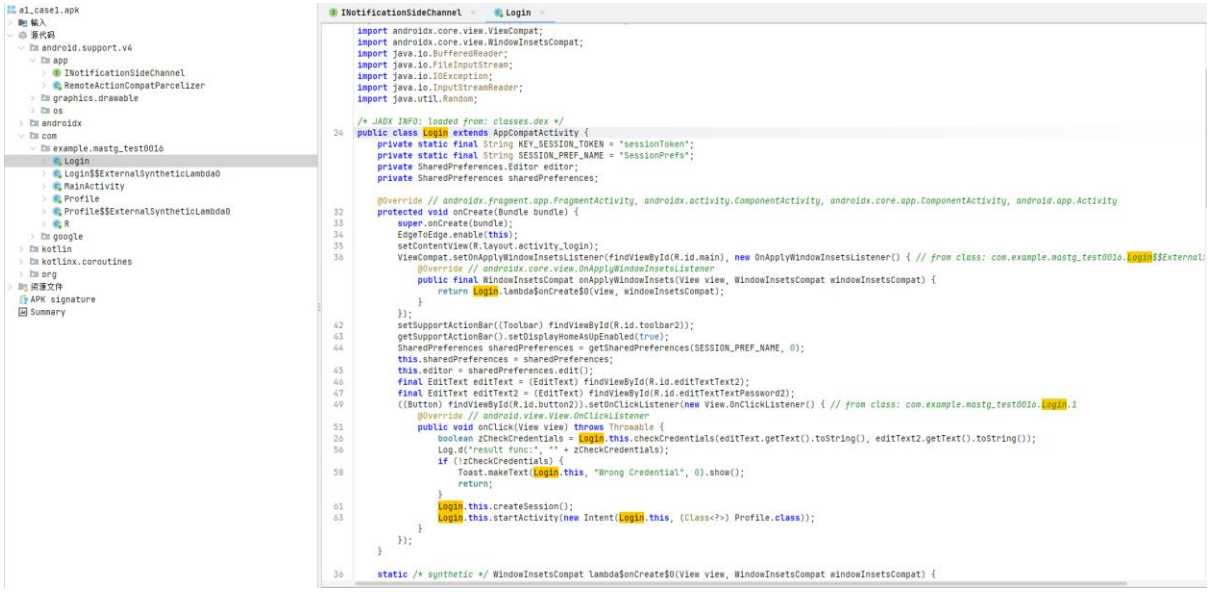
1. Launch `jadx-gui.bat`
2. Drag the APK into the window, or use `File > Open`
3. Browse the decompiled Java and resources
4. Use `File > Save All` if you want the output written to disk

Decompile an APK with the command line

From PowerShell in the `bin` folder:

```powershell
.\jadx.bat -d C:\output\myapp C:\path\to\app.apk
```

   • Decompile the APK and confirm you can access package name, main activity, manifest, and at least one login/registration class. • Include one screenshot of a relevant decompiled class (for example, login logic).

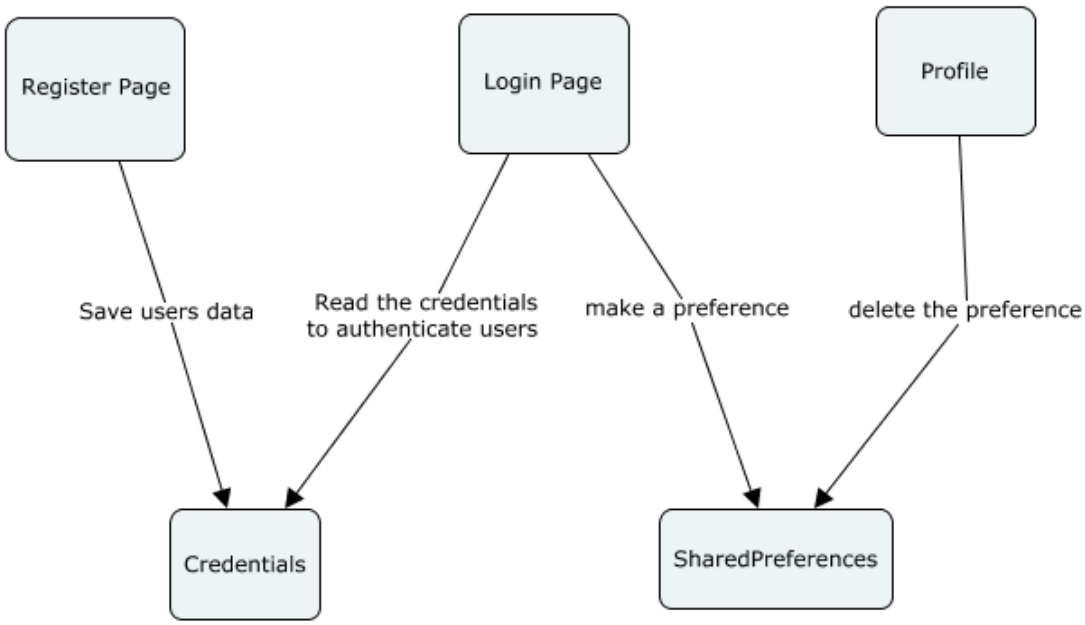


```
a1_case1.apk
  输入
  源代码
    android.support.v4
      app
        INotificationSideChannel
        RemoteActionCompatParcelizer
      graphics.drawable
      os
    androidx
    com
      example.mastg_test0016
        Login
        Login$$ExternalSyntheticLambda0
        MainActivity
        Profile
        Profile$$ExternalSyntheticLambda0
        R
      google
    kotlin
    kotlinx.coroutines
    org
  资源文件
  APK signature
  Summary
```

```
INotificationSideChannel   Login
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.Random;

/* JADX INFO: loaded from: classes.dex */
public class Login extends AppCompatActivity {
    private static final String KEY_SESSION_TOKEN = "sessionToken";
    private static final String SESSION_PREF_NAME = "SessionPrefs";
    private SharedPreferences.Editor editor;
    private SharedPreferences sharedPreferences;

    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
    protected void onCreate(Bundle bundle) {
        super.onCreate(bundle);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_login);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), new OnApplyWindowInsetsListener() { // from class: com.example.mastg_test0016.Login$$External
            @Override // androidx.core.view.OnApplyWindowInsetsListener
            public final WindowInsetsCompat onApplyWindowInsets(View view, WindowInsetsCompat windowInsetsCompat) {
                return Login.lambda$onCreate$0(view, windowInsetsCompat);
            }
        });
        setSupportActionBar((Toolbar) findViewById(R.id.toolbar2));
        getSupportActionBar().setDisplayHomeAsUpEnabled(true);
        SharedPreferences sharedPreferences = getSharedPreferences(SESSION_PREF_NAME, 0);
        this.sharedPreferences = sharedPreferences;
        this.editor = sharedPreferences.edit();
        final EditText editText = (EditText) findViewById(R.id.editTextText2);
        final EditText editText2 = (EditText) findViewById(R.id.editTextTextPassword2);
        ((Button) findViewById(R.id.button2)).setOnClickListener(new View.OnClickListener() { // from class: com.example.mastg_test0016.Login.1
            @Override // android.view.View.OnClickListener
            public void onClick(View view) throws Throwable {
                boolean zCheckCredentials = Login.this.checkCredentials(editText.getText().toString(), editText2.getText().toString());
                Log.d("result func:", " " + zCheckCredentials);
                if (!zCheckCredentials) {
                    Toast.makeText(Login.this, "Wrong Credential", 0).show();
                    return;
                }
                Login.this.createSession();
                Login.this.startActivity(new Intent(Login.this, (Class<?>) Profile.class));
            }
        });
    }

    static /* synthetic */ WindowInsetsCompat lambda$onCreate$0(View view, WindowInsetsCompat windowInsetsCompat) {
```

2. Task 2: Understand the App and Build a Simple Model • Ideally, run the app in an emulator/device and interact with it (register, log in, log out). If not possible, use tutor demos and static analysis.

• In your own words, summarise the app's purpose and main features in 3–5 sentences.
The app is used to save users' profiles. It has a page for login and another page for registering. After user login, they can see their profiles in a new page.
• Draw a simple system model showing main components (e.g., UI, storage, external services), important assets (e.g., credentials/tokens), and data flows.



• State core assumptions and attacker goals (what can be observed/controlled and what the attacker aims to achieve).
We assume the app is running in a standard android environment; the attacker can record the time when preference is made. The attacker's goal is to get the users' profiles.

3. Task 3: Find at Least One Vulnerability (Hint: Randomness and Crypto)
• Use static analysis to locate all values treated as "random", "codes", "tokens", or similar.
• For each location, record class/method, generator API/helper, and security role (UI-only, identifier, token, key, etc.).

MainActivity.randomNumberGenerator() uses new Random().nextInt(100) at MainActivity.java (line 17). Generator: java.util.Random. Role: non-security UI/debug value (logged), not used in auth.

Login.generateSessionToken() builds a 16-char alphanumeric token using Random.nextInt(62) at Login.java (line 183), stored in session prefs at Login.java (line 175). Generator: java.util.Random. Role: session/auth token.

Session token lifecycle helpers exist in Login.java (line 179) and Profile.java (line 49).

Role: token storage/removal.

• With AI help if needed, assess whether any generator/pattern is unsuitable for security-sensitive values (for example, predictable outputs). • Select at least one security-relevant weak usage (for example, session/authentication be- havior) as your core vulnerability.

The core vulnerability is java.util.Random . It is predictable and unsuitable for security-sensitive session tokens, and the token is not actually validated before protected navigation (Login launches Profile directly at Login.java (line 59)). This makes session behavior weak and easy to bypass/forge in a tampered client context.

4. Task4: Explain What Went Wrong and the Threat Model
   • Explain where the value is generated, why the method is unsuitable for its security role, and what attacker knowledge/observation is needed to exploit it.
   Session token is generated in `Login#generateSessionToken`. Outputs from `java.util.Random` are predictable from seed/state observations. For token prediction: attacker needs approximate token creation time and one or more observed outputs to narrow seed/state.
   For practical bypass in this app: attacker only needs local app-data access to read/write SharedPreferences or call Activities directly.

   • Describe a realistic attacker model (e.g., network observer, another app, rooted-device user) and concrete capabilities.
   The goal of attacker is to get the profile; their capabilities include reading and modifying SessionPrefs.xml and they can repackage the APK to skip checks or auto-navigate to privileged screens.

   • Provide a short step-by-step attack scenario (for example, prediction/influence leading to auth bypass or impersonation).
   Attackers install app on rooted/emulated test device and launch it once. Then they inspect `SessionPrefs.xml` and write any `sessionToken` value. Finally app opens `Profile` without verifying token validity/presence, so attacker reaches post-login UI state.

5. Task 5: Write Up Your Findings • Use the official USENIX paper template (LaTeX or Word): https://www.usenix.org/ conferences/author-resources/paper-templates. Limit the report to 2 pages max, including figures/references. • Within those 2 pages, include the following content (you may group headings as you wish, but all items must be covered): – Short introduction (goal and why APK analysis is used). – System/threat model (protected

assets, attacker, capabilities). – How you found the vulnerability (tooling and AI use, if any). – Vulnerability details (code location, risk, exploitation path). – Concrete fix/mitigation and why it works. • Add an AI usage log (summaries acceptable) as a separate file; include references for tools/docs used. • Before submission, run at least one rubric-driven LLM mock Q&A. Paste the current rubric, answer follow-up questions with evidence, and record key questions/improvements in the AI log. • Prepare a recorded presentation (≤ 5 minutes) summarising model, vulnerability, and key lessons.